

TDD - Din tur til å eksperimentere for kartlegging av behov!

Info:

Etter de første øktene i Testdrevet utvikling (TDD) har dere lært hvordan tester skrives og hvordan kode kan utvikles gjennom små iterasjoner.

I denne oppgaven skal dere bruke TDD som et verktøy for å utforske et problemområde. Målet er ikke å designe hele løsningen på forhånd, men å la testene hjelpe dere med å oppdage hvilke egenskaper systemet trenger. **Ikke planlegg hele systemet på forhånd.**

Målet med øvelsen er å bruke testene til å oppdage hvordan systemet bør oppføre seg. Dersom du oppdager nye krav underveis, er det et tegn på at du bruker TDD slik det er ment.

Prosjektet dere starter på nå vil videreutvikles gjennom resten av modulen og gradvis vokse til et komplett system.

Velg ett av følgende prosjekter:

- Poengsystem (Nybegynnervennlig)
- XP / Level-system (Intermediate)
- Bibliotekssystem (Utfordrende - begrenset scope)

Del 1 – Utforsk behovet for minimum viable product (MVP)

Bruk Excalidraw til å skissere hvordan du ser for deg en minimal versjon av systemet (MVP). Du trenger ikke designe databaser, API-er eller tekniske detaljer.

Fokuser på spørsmål som:

- Hvilket problem løser systemet i sin enkleste form?
- Hvem er “brukeren” i denne første versjonen?
- Hva er den eneste kjerneflyten systemet må støtte?
- Hvilke objekter trengs kun for denne flyten?
- Hva er absolutt minimum for at systemet “gir mening”?

Begrensning:

Systemet skal kun ha én hovedflyt (end-to-end) i MVP.

Alt annet er “fremtidig utvidelse” og kan tegnes separat hvis ønsket i praksisforberedende.

Sett opp prosjektet på VSC

Følg for å sette opp prosjektet:

<https://github.com/Joviank/IntroTDDOppgave/blob/master/SetteOppProsjektMedxUnit.txt>

Struktur kommer til å være en viktig del av modulen, og dermed anbefales denne strukturen:

```
Din-valgte-prosjekt
|
├─ DinOppgave.sln
|
├─ DinOppgave
|   ├─ DinOppgave.cs
|   └─ DinOppgave.csproj
|
└─ DinOppgave.Tests
    ├─ DinOppgaveTests.cs
    └─ DinOppgave.Tests.csproj
```

Del 2 – Formuler hypoteser gjennom tester

Skriv tester som beskriver oppførselen du ønsker. Start med tester som beskriver den absolutt enkleste versjonen av systemet. Hvis en test ikke direkte støtter MVP-flyten, skal den ikke skrives.

Eksempler:

Poengsystem

- En bruker kan få poeng
- En bruker starter med 0 poeng
- Poengsummen øker når poeng legges til

XP / Level-system

- En spiller starter på level 1
- XP kan legges til
- Spillere går opp i level når nok XP er oppnådd

Bibliotekssystem

- En bok kan registreres
- En bok kan lånes ut
- En utlånt bok kan ikke lånes ut på nytt

Systemet skal ikke utvides utover MVP-kjernen.

Det betyr ingen ekstra features, moduser eller “forbedringer” utover det testene beskriver.

Du skal kun skrive **maks 4–5 tester totalt (inkludert eksemplene)**

Ikke mer. Hvis du merker at du trenger mange ekstra tester, er det et tegn på at du bør stoppe og holde deg til kjernen.

Del 3 – Implementer minimumskoden

Implementer kun nok kode til at testene passerer.

Følg TDD-syklusen:

1. Skriv en test
2. Se testen feile
3. Skriv minimumskoden som får testen til å passere
4. Refaktorer

Del 4 - Innlevering

Dere skal publisere repoet på Github, og dere leverer inn kun en lenke

<https://github.com/DittGithubBrukernavn/DinValgteOppgave-prosjekt>

Dette er hva som skal leveres:

- GitHub-lenke til repo
- Excalidraw-skisse

Struktur i repo:

- Kode
- Tester
- Excalidraw.png

Veien videre

For kommende oppgaver i modulen skal dere commite til github-repoet. Dette er for å holde Githuben deres ryddig, men også for å gi dere trening i å skrive commit meldinger.

Har du noen spørsmål eller trenger hjelp?

Ikke nøl med å ta kontakt med ansvarlig fagveileder på github. (:

Masse lykke til!